Analizador Léxico

Rocío Corral[1]

[1]Lic. Cs. de la Computación/Facultad de Ingeniería/UNCuyo

21 de Marzo de 2023

Abstract

Proceso de construcción de un analizador léxico para el lenguaje de programación TinyRust+. Decisiones de diseño, metodología, lenguaje utilizado, arquitectura, funcionamiento y resultados se explican en el presente trabajo.

1. Introducción

El objetivo de este informe es explicar y fundamentar la implementación de la primera etapa del compilador de TinyRust+. En esta primera etapa se implementó el analizador léxico, programa que en un futuro será el encargado de proveer al analizador sintáctico un conjunto de *tokens* generados a partir de un archivo de texto. Estos tokens se definen utilizando como base las reglas descritas en el manual de TinyRust+ para delimitar palabras reservadas, operadores, identificadores, tipos y demás.

2. Metodología

Para el desarrollo de este proyecto, el primer paso fue hacer un análisis de la información disponible sobre el funcionamiento esperado de TinyRust+ y definir una serie de aspectos de diseño detallados en la siguiente sección. Posterior a esto, se diagramó la arquitectura del proyecto y finalmente se implementó.

A la hora de la implementación, se trabajó con el lenguaje de programación Java. Se utilizó Git junto con GitHub como herramienta de control de versiones, de manera tal que ambos integrantes del equipo pudieran trabajar colaborativa e independientemente en el proyecto de manera simultánea.

2.1 Consideraciones

A continuación se listan una serie de consideraciones y decisiones de diseño relevantes antes de profundizar en la implementación del compilador.

El alfabeto de entrada del compilador consta de los caracteres con código ASCII desde 32 hasta 126[1]. Adicionalmente, también se incluyen los caracteres:

- Salto de línea (ASCII 10)
- Tabulación (ASCII 9)
- Fin de texto (ASCII 3)
- Tabulación vertical (ASCII 11)

En la Tabla 1 se encuentra definido por extensión el alfabeto de TinyRust+.

[space]	!	"	#	$	%	&	'	(	)	*	+	,	-	.	0	1	2	3	4
[tab]	5	6	7	8	9	:	;	<	=	>	?	@	A	B	C	D	E	F	G
[vtab]	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
[break]	[	]	_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
[etx]	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	\	/	^	

Tabla 1. Alfabeto de entrada de TinyRust+.

Cabe destacar que ciertos caracteres exclusivos del español, como “ñ” y vocales con tilde (tanto mayúsculas como minúsculas) no forman parte del alfabeto de entrada de este compilador.

En las tablas 2 y 3 se puede apreciar la lista de tokens que el analizador léxico reconoce. En cada tabla se encuentra el tipo del token acompañado de la expresión regular que lo representa. La Tabla 2 muestra la lista de tokens de palabras reservadas del lenguaje, mientras que la Tabla 3 contiene todos los demás tokens reconocidos tales como literales, operadores, comentarios y delimitadores.

Tipo de token	Expresión regular
p_class	class
p_else	else
p_false	false
p_if	if
p_return	return
p_while	while
p_true	true
p_nil	nil
p_false	false
p_new	new
p_func	fn

Tipo de token	Expresión regular
p_static	static
p_pub	pub
p_void	void

Tabla 2. Tokens de palabras reservadas.

Tipo de token	Expresión Regular	Notas
id	(a..z + _)(a..z + A..Z + _ + 0..9)*	Identificador: comienzan con una letra minúscula o guión bajo y luego pueden contener cualquier cantidad de letras minúsculas, mayúsculas, números o guiones bajos. Exceptuando las palabras reservadas.
lit_int	$(0..9)^+$	Literal entero: cualquier sucesión de dígitos numéricos.
lit_string	“Σ*”	Literal cadena: contiene cualquier cantidad de caracteres del alfabeto de entrada encerrados entre comillas dobles.
lit_char	‘(Σ + \Σ)’	Literal caracter: contiene posiblemente un backslash y luego un único caracter del alfabeto de entrada encerrado entre comillas simples.
type	(A..Z)(a..z + A..Z + _ + 0..9)*	Tipo de dato: similar a un identificador, solo que comienza con una letra mayúscula.
multi_line_comment	/* Σ* */	Comentario multilínea: puede contener cualquier cantidad y tipo de caracteres (incluyendo saltos de línea) encerrados entre ambas secuencias de apertura y cierre.
single_line_comment	//Σ* ([break] + [etx])	Comentario de una línea: comienza con un par de slashes y puede contener cualquier cantidad y tipo de caracteres, finalizando en un salto de línea (o final del archivo).
p_true	true	
p_false	false	

op_add	+	
op_sub	-	
op_prod	*	
op_div	/	
op_mod	%	
op_or	||	
op_and	&&	
op_less	<	
op_great	>	
op_less_eq	<=	
op_great_eq	>=	
op_equal	==	
assignment	=	
semicolon	;	
colon	:	
open_par	(	
close_par	)	
open_curly	{	
close_curly	}	
open_bracket	[	
close_bracket	]	
return_type	->	

op_not	!	
op_not_eq	!=	
comma	,	
dot	.	

Tabla 3. Tokens de literales, operadores, comentarios y delimitadores.

2.2 Errores

Los posibles errores que fueron previstos en la etapa de diseño del analizador léxico fueron:

- *Identificador mal formado*: error generado al encontrar un lexema que es cercano a un identificador pero no cumple con su sintaxis por algún motivo. Por ejemplo, la cadena '123var' comienza con dígitos seguidos de letras, por lo cual no es ni un literal entero ni un identificador. Otro ejemplo podría ser una cadena que comienza con minúsculas y, sin un espacio de por medio, tiene un "@", por ejemplo, "vari@ble".
- *Comentario sin cerrar*: error generado al encontrar una secuencia de apertura de comentario multilínea y luego llegar al final del archivo sin encontrar la secuencia de cierre, es decir, "*/".
- *Símbolo no perteneciente al alfabeto*: error generado al leer en el archivo de entrada un caracter que no esté incluido en el alfabeto Σ, como por ejemplo: "¬", "ñ", "á", etc.

Luego, durante la implementación, surgieron nuevos posibles errores para manejar:

- *Token desconocido*: error generado al encontrar un lexema que no cumple ningún patrón reconocible, por ejemplo, "|". Cabe destacar que "||" sí es reconocido como el operador lógico OR. Sin embargo, "|" por sí solo, no es un token de este compilador.
- *Literal inválido*: error generado al hallar literales de tipo cadena o caracter con una estructura inválida. Por ejemplo, literal cadena sin cerrar o literal caracter conteniendo más de un caracter.

3. Resultados

3.1 Estructura general

El repositorio del proyecto se encuentra estructurado en una jerarquía de directorios para facilitar su legibilidad. En la Figura 1 puede apreciarse esta jerarquía.

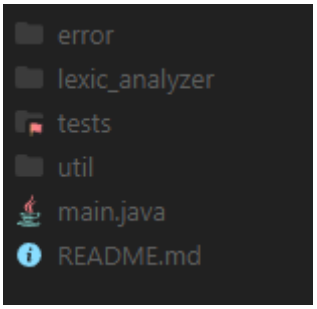


Figura 1. Jerarquía de carpetas

En el directorio raíz se encuentran los siguientes archivos y directorios:

- Archivo *main.java*: contiene la clase TinyRustCompiler, el punto de entrada del compilador.
- Carpeta *util*: contiene clases que proveen algún tipo de asistencia mediante métodos. Por ejemplo: la clase Logger, que provee métodos necesarios para escribir mensajes en el archivo de salida.
- Carpeta *tests*: contiene los archivos .rs de prueba utilizados para probar el analizador léxico. Cada archivo de prueba incluye el output esperado al analizarlo.
- Carpeta *lexic_analizer:* contiene todas las clases relacionadas al analizador léxico de nuestro compilador. Incluye al analizador léxico, el ejecutor y las clases Token y ReservedWords.
- Carpeta *error*: contiene una clase para cada posible error que pueda ser arrojado al momento de ejecutar el compilador de TinyRust+

3.2 **Implementación**

En esta sección se ahondará en la implementación del analizador léxico de TinyRust+, explicando el rol y funcionamiento de cada una de las clases que lo componen.

3.2.1 **TinyRustCompiler**

Esta es la clase raíz del compilador. Recibe los argumentos de la línea de comandos para los archivos de entrada y salida y se encarga de guardarlos. Además, en el método main de esta clase se instancia la clase *Executor* y se invoca su método *run* para comenzar la ejecución del compilador.

3.2.2 **Executor**

Esta consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase LexicAnalizer que recibe como parámetro la ruta del archivo de entrada. Una vez instanciado el analizador léxico, el ejecutor llamará iterativamente al método *nextToken* del LexicAnalyzer, el cual devuelve el siguiente token encontrado en el archivo de entrada. Al recibir el token, lo agrega a una lista de *Token,* la cual será mostrada en el archivo de salida o por pantalla en el caso de tener un análisis léxico

exitoso. Caso contrario, la salida mostrará en qué línea, columna y token se presentó el error.

3.2.3 **LexicAnalizer**

Esta clase es la encargada de todo el proceso de análisis léxico. En su constructor se abre el archivo de entrada e inicializan contadores de línea y columna.

LexicAnalizer cuenta con diversos métodos que permiten el correcto análisis léxico de un archivo de texto. Provee únicamente dos métodos públicos, que se describen a continuación:

nextToken: consume parte del archivo de entrada para reconocer y retornar el siguiente token. En caso de haber consumido todo el archivo, devuelve el token EOF (end of file). Si ocurriera algún error al intentar decodificar el archivo, este método arrojaría un error del tipo *LexicalError.*

hasNextToken: verifica si ya se ha alcanzado el final del archivo y devuelve un valor booleano acorde. Este método puede invocarse como validación antes de invocar a *nextToken*. Siempre que *hasNextToken* devuelva *false*, está garantizado que *nextToken* devolverá el token EOF (end of file).

A su vez, *LexicAnalyzer* implementa una gran cantidad de métodos privados que son utilizados directa o indirectamente por el método *nextToken*. Los métodos privados de la clase se listan y describen a continuación.

isValidChar: recibe una variable de tipo char y verifica si este caracter pertenece al alfabeto de entrada.

isLowercaseChar: recibe una variable de tipo char y verifica si se encuentra en el rango de los caracteres en minúscula.

isUppercaseChar: análogo a *isLowercaseChar*, este método recibe un caracter y determina si es mayúscula.

isAlphabet: recibe una variable char y determina si es una letra del abecedario (alfabeto español).

isDigit: recibe una variable char y comprueba si es un dígito numérico.

isOperator: recibe una variable char y determina si es un operador o algún delimitador.

isBlankSpace: recibe una variable char y comprueba si se trata de algún caracter de espacio en blanco, tales como saltos de línea, tabulaciones, espacios y final de archivo.

isLineBreak: recibe una variable char y determina si se trata de un salto de línea o final de archivo.

matchIntLiteral: recibe una variable char y una variable de tipo *Token*. A partir del caracter inicial recibido, verifica si lo que le sigue es un literal numérico entero. Devuelve true si el token analizado es literal entero y actualiza los atributos del token recibido. Si el lexema estuviera seguido (sin espacios de por medio) por un token identificador, este método arroja un error de tipo *BadIdentifierError*.

matchStringLiteral: recibe una variable char y una variable de tipo *Token*. A partir del caracter inicial, se encarga de detectar si el token actual es un literal de cadena. Devuelve true si ese fuera el caso y actualiza los atributos del token recibido. Si encuentra un *nil* o salto de línea, arroja un error de tipo *InvalidLiteralError*.

matchCharLiteral: análogo a *matchStringLiteral.* Este método se encarga de detectar si el token que está siendo analizado corresponde a un único caracter encerrado entre comillas simples. Si encuentra un *nil* o salto de línea, arroja un error de tipo *InvalidLiteralError*.

matchIdentifier: recibe una variable char y una variable de tipo *Token*. Su comportamiento es como el de un autómata finito determinista con 4 estados. Este se encarga de verificar que el identificador comience con un caracter del alfabeto español (exceptuando la ñ) o con un guión bajo. Si es así, continúa consumiendo caracteres siempre y cuando sean dígitos, guiones bajos o letras del abecedario. Si encuentra un espacio en blanco, un fin de archivo, o algún operador listado en el método *isOperator* frena la ejecución y acepta la cadena de entrada como un token de tipo identificador. En caso de tratarse de un identificador mal formado (en caso de incluir caracteres del alfabeto no válidos para un identificador), arroja el error de tipo *BadIdentifierError*.

matchTypeIdentifier: este método recibe un token de tipo identificador y verifica que el lexema del identificador encontrado comience con mayúscula. De ser así, el identificador pasa a ser identificador de tipo.

matchMultilineComment: recibe una variable char y una variable tipo *Token*. Actúa como un autómata finito determinista de 5 estados. Verifica que la cadena encontrada comience con “/*”. Si es así, consume cualquier caracter perteneciente al alfabeto de entrada hasta encontrar la secuencia de cierre, “*/”. Si se encuentra con el fin del archivo antes de encontrar el cierre del comentario multilínea, arroja un error de tipo *UnclosedMultilineCommentError*. Al ser un comentario, al identificarse este token no se agrega a la lista de tokens para el analizador sintáctico, sino que la función *nextToken* procede a buscar de forma recursiva el próximo token que no sea un comentario.

matchComment: recibe una variable char y una variable tipo *Token*. Actúa como un

autómata finito determinista de 4 estados. Verifica si el token que se está analizando comienza con "//". De ser así, consume cualquier caracter perteneciente al alfabeto de entrada hasta que se encuentra con un salto de línea o el final del archivo. Igual que con el comentario multilinea, al identificarse este token no se agrega a la lista de tokens para el analizador sintáctico, sino que la función *nextToken* procede a buscar de forma recursiva el próximo token que no sea un comentario.

consumeSpaces: se encarga de descartar espacios en blanco del archivo de entrada. Arroja *InvalidCharacterError* si se encuentra con algún caracter no perteneciente al alfabeto de entrada

readConsumeChar: lee el siguiente caracter del archivo, lo consume y aumenta el contador de columna. Es decir, una vez leído no hay vuelta atrás. Si se encuentra con el fin de una línea, verifica si hay una línea más. De ser así, incrementa el contador de línea, reinicia el contador de columna y continúa leyendo la siguiente línea del archivo. Si encuentra el final del archivo, marca que se llegó al final de la lectura y cierra el archivo.

readWithoutConsumeChar: análogo a *readConsumeChar*, lee el siguiente caracter del archivo pero no lo consume. Es decir, no se pierde el caracter que consumido. Si no se invocara a la función *readConsumeChar*, la función *readWithoutConsumeChar* devolvería siempre el mismo caracter en llamados sucesivos.

3.2.4 **Token**

Esta clase representa cualquier token perteneciente a TinyRust+. Cada instancia guarda el número de línea y columna en donde fue encontrado dicho token junto a su lexema y tipo. Al ser todas sus variables privadas, esta clase cuenta con los getters y setters necesarios para accederlas y modificarlas.

3.2.5 **ReservedWords**

Esta clase estática contiene un único método estático que decide si un identificador es una palabra reservada. Su método *matchReservedWord* recibe una variable de tipo *Token* y compara su lexema contra la lista de palabras reservadas de TinyRust+. Si este presenta coincidencia con alguna palabra reservada, entonces se actualiza el tipo del token con el de la palabra reservada correspondiente.

3.3 **Errores**

Para el manejo de errores del analizador léxico se implementó la clase abstracta *LexicalError,* que hereda de la clase *Exception* de Java. Un error de esta clase espera recibir el número de línea y columna donde ocurrió el error junto con una descripción. A la hora de mostrar el mensaje de error, este incluirá estos tres datos.

Para cada tipo de error distinto que pueda ocurrir durante el análisis léxico, se creó

una clase distinta que hereda de LexicalError. A continuación se listan estas clases y se explica brevemente bajo qué condición se arroja cada excepción.

- *BadIdentifier*: si se encuentra un identificador con caracteres inválidos.
- *InvalidCharacter*: si el archivo de entrada contiene un caracter que no pertenece al alfabeto de entrada de TinyRust+.
- *UnclosedMultiLineComment*: si se encuentra una apertura de comentario multilínea y se llega al final del archivo antes de encontrar el cierre del comentario.
- *UnmatchedToken*: si se encuentra una secuencia de caracteres que no coincide con ningún token de TinyRust+. Por ejemplo: “&”, “|”, “@var”, etc.
- *InvalidLiteral*: si se encuentran literales cadena, caracter o entero mal formados. Por ejemplo, un literal entero de más de 32 bits, un literal caracter con más de un caracter, una cadena con un caracter nulo o un literal caracter o cadena sin cerrar.

3.4 Casos de prueba

En el código del proyecto se incluyen los casos de prueba que se utilizaron para poner a prueba el funcionamiento del analizador léxico. La lista consiste en más de 20 archivos de prueba para casos exitosos y casos de fallo. Dichos archivos se pueden encontrar en el directorio *tests* del proyecto.

Particularmente el archivo de test número 0 corresponde al ejemplo de código de TinyRust+ provisto por la cátedra en el manual del lenguaje. El análisis léxico de este archivo es exitoso.

El archivo de test número 1 contiene también una gran cantidad de tokens válidos del TinyRust+ para poner a prueba que sean reconocidos correctamente.

Los archivos de tests 2 y 3 no contienen código y son casos exitosos también, dado que el analizador léxico no debería arrojar ningún error si recibe archivos vacíos.

Mientras que el resto de archivos de prueba son casos de falla, en los que se espera que el compilador arroje cierto error en un token en particular. Ejemplos de estos casos de prueba son: identificadores mal formados, caracteres inválidos, cadenas sin cerrar, comentarios multilínea sin cerrar, literales de tipo carácter con muchos caracteres, literales cadena o carácter que incluyen nil, entre otros.

3.5 Compilado y ejecución

Para compilar el proyecto, deben ejecutarse los siguientes comandos desde el directorio raíz del proyecto.

Paso 1: compilar.

```
javac error/*.java lexic_analyzer/*.java util/*.java *.java
```

Paso 2: generar el jar.

```
jar cfm etapa1.jar Manifest.txt *
```

Para ejecutar el proyecto debe ejecutarse el siguiente comando:

```
java -jar etapa1.jar <ARCHIVO_FUENTE> [<ARCHIVO_SALIDA>]
```

Es obligatorio especificar la ruta del archivo fuente que TinyRust+ analizará. En el directorio *tests* se pueden hallar archivos de prueba para alimentar el comando. Por ejemplo:

```
java -jar etapa1.jar ./tests/"test (1).rs" ./output/example.o
```

Especificar un archivo de salida es opcional. En caso de ser especificado, se escribirá la salida del compilador en dicho archivo. Si el archivo existe previamente será sobreescrito, caso contrario será creado por el compilador. En caso de no especificarse este parámetro, la salida se mostrará por pantalla.

4. Conclusiones

Comenzar el camino de la implementación de un compilador resultó ser una experiencia sorprendente donde pudimos aplicar conocimientos de varias materias. Este proceso requiere conocimientos consolidados de variadas materias como Lenguajes Formales, al pensar e implementar algunos métodos como autómatas finitos deterministas y Paradigmas de Programación al trabajar con el paradigma orientado a objetos u otros paradigmas que pudiera implementar nuestro compilador.

Durante las próximas etapas de desarrollo, este proyecto se irá complejizando cada vez más, pero tenemos la expectativa de que las bases construidas en esta primera etapa sirvan de sólidos cimientos para lo que se construirá encima.

5. Referencias

1. ASCII-CODE. ASCII table according to Windows-1252. https://www.ascii-code.com/